

Polyglot's Shorthand

Linguistically-Grounded Normalization of
Romanized / Code-Mixed South Asian Text

Inter-IIT Tech Meet 2026 — Problem 4

Constraint: $\leq 500\text{M}$ parameters, single-digit-ms latency

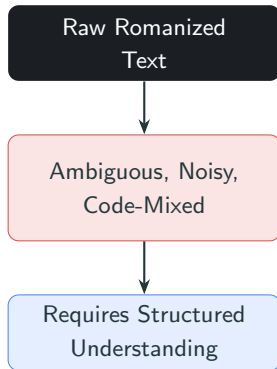
The Problem

Efficient linguistic processing of noisy Romanized / code-mixed South Asian text,
under a strict deployment constraint:

- $\leq 500\text{M}$ parameters
- single-digit-millisecond latency

"bhai order cancel krdo please, urgent meeting h"

"item delivered bol rha h but mila hi nhi"



Why Romanized Text Is Hard

Romanization noise is **not simply random noise**:

- Multiple spellings can represent the **same** underlying content

kar rahe ho / karre ho / kre ho

- Some spelling differences **change lexical identity**

din (day) vs deen (religion) — sadi vs shadi (marriage)

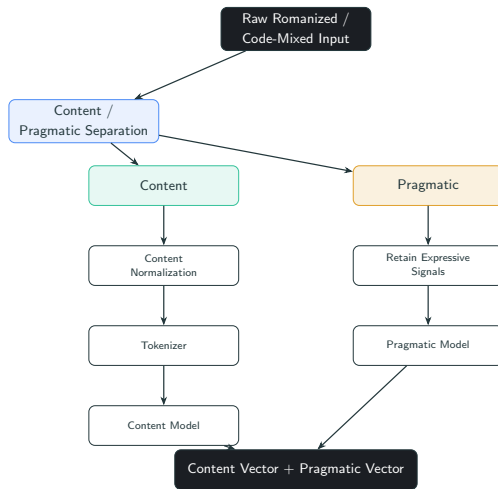
- Emojis, punctuation, repetition and slang carry **pragmatic meaning**

soooo good $\neq$ so good

- **Context** may be required to resolve ambiguity

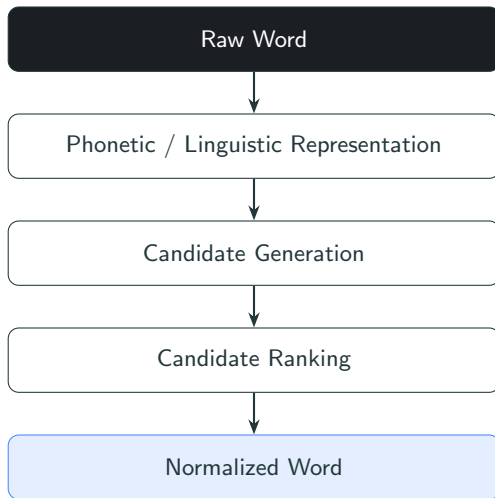
humne vs hamane — resolved only by sentence context

Our Representation: Content vs Pragmatics



Context is **not** a persistent third vector — retrieved only when content + pragmatic signals are insufficient.

Planned Normalization Pipeline



Guiding principle

Normalize irrelevant surface variation while **preserving distinctions that can change lexical identity.**

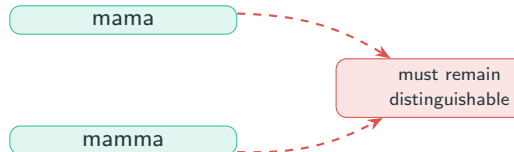
Phonetic Normalization Hypothesis

We hypothesized Romanized Hindi should be handled via **linguistic / phonetic structure**, not arbitrary character noise.

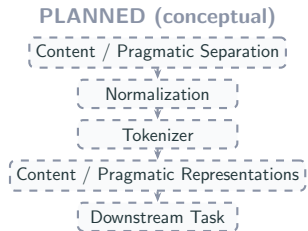
Minimal-pair stress test

mama vs *mamma*

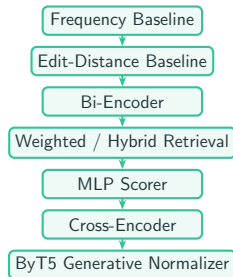
Do **not** over-normalize repeated characters and destroy lexical information — naive collapsing of doubled letters can merge genuinely distinct words.



Planned Architecture vs. Implemented Experiments



IMPLEMENTED / TESTED



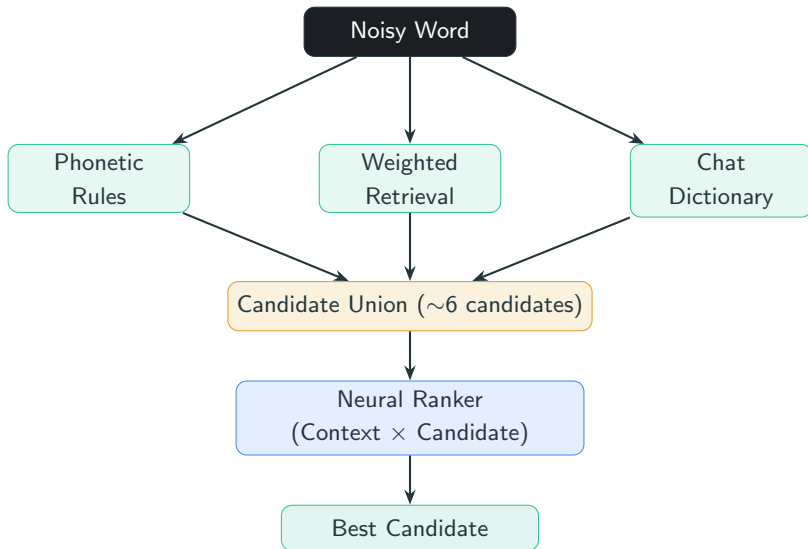
Solid = experimentally implemented and measured. Dashed = conceptual / not yet built.

Experimental Evolution



- Each step is a **controlled experiment**, not a guess — every change was measured against the previous baseline.
- Progression driven by evidence: candidate generation vs. ranking vs. representation vs. generation.

Candidate Retrieval Architecture (Hybrid V1)



Retrieval Bottleneck

47.61%

Hybrid V1 candidate recall

35.69%

Hybrid V1 end-to-end Recall@1

Key finding

If the gold word is never retrieved, **no ranker can recover it**. The retriever — not the ranker — is the dominant source of error.

Ranker variant	Ranker accuracy*	End-to-end Recall@1
Hybrid V1 (dot-product)	76.92%	35.69%
Hybrid + MLP scorer	76.70%	35.59%
Hybrid + Cross-Encoder	50.36%	23.37%

*accuracy conditioned on gold being present in the candidate set. Adding ranker capacity did not improve — and Cross-Encoder degraded — results.

Generative Normalization with ByT5

Model: google/byt5-small

Parameters: 299,637,760

Byte-level encoder-decoder — robust to arbitrary spelling without a fixed vocabulary.

Targeted span generation: full sentence as context, only the marked span is normalized.

Input

normalize: hello <n>wrld</n>!

Target

world

Model generates **only** the normalized word, not the full sentence.

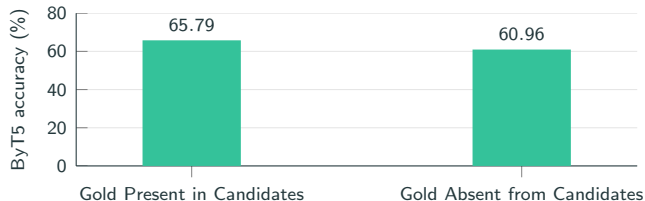
Dataset	239,475 sentence-level instances (COMI-LINGUA)	
Split	80 / 10 / 10 (Test $N = 23,948$)	
Test distribution	Noisy = 4,767	Clean = 19,181

Results

Approach	Metric	Value	Notes
Edit-distance baseline	Recall@1	88.29%	simple string baseline
Original V1 Bi-Encoder	Recall@1	95.02%	easier distribution, not comparable
Hybrid V1	End-to-end Recall@1	35.69%	candidate recall = 47.61%
ByT5 — overall	Exact match	90.22%	N = 23,948
ByT5 — noisy words	Exact match	63.48%	N = 4,767
ByT5 — clean preservation	Exact match	96.87%	N = 19,181

95.02% is **not** the final system accuracy — it is the Original V1 result on its original, easier candidate distribution. Hybrid V1's final end-to-end Recall@1 is **35.69%**.

Retrieval-Missed Cases — Key Finding

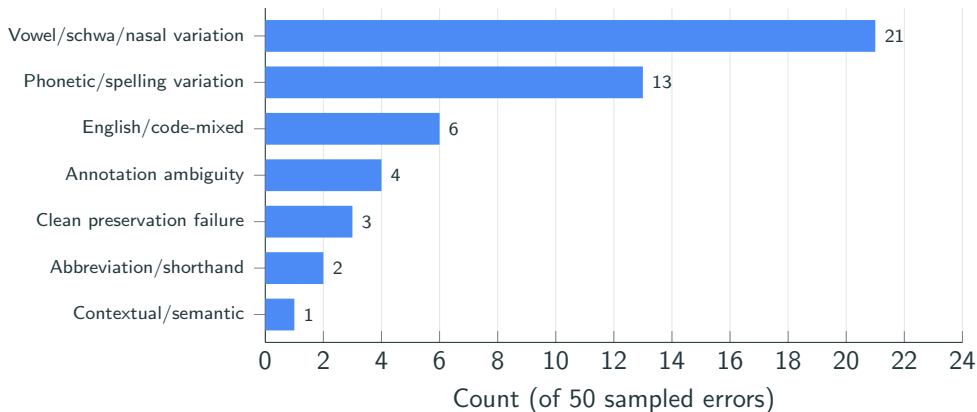


Condition	ByT5 accuracy	<i>N</i>
Gold present in Hybrid V1 candidates	65.79%	2,485
Gold absent from Hybrid V1 candidates	60.96%	2,282

Interpretation

If the gold word is absent from a candidate set, a candidate ranker **cannot** select it. ByT5 achieved 60.96% exact match on retrieval-missed cases — it demonstrates the ability to bypass candidate-generation limitations.

Error Analysis — ByT5 (50 sampled errors)



Majority of errors (34/50) are near-miss phonetic/vowel variants, not unrelated outputs.

Accuracy vs. Latency Tradeoff

Stage	Time
Tokenization	2.52 ms
Encoder + decoder	39.43 ms
Decoding	24.81 ms
End-to-end	66.76 ms
P50	63.89 ms
P95	95.81 ms

Batch size 1, RTX 5070 Ti, 500 timed iterations, model loading excluded.

Constraint violated

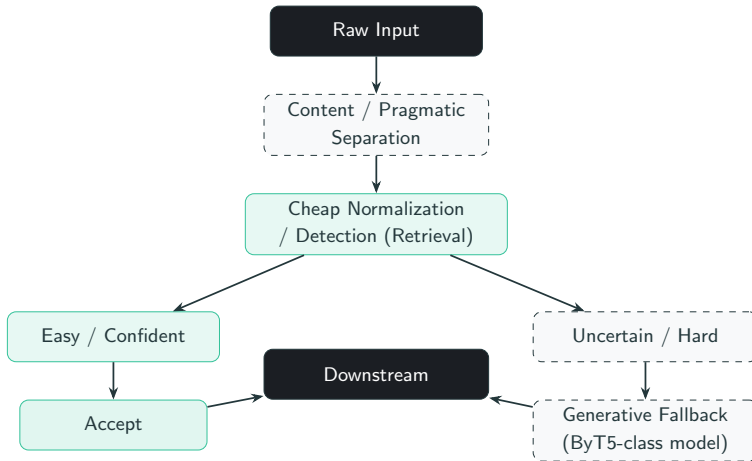
The competition requires **single-digit-ms** latency.

ByT5 does **not** meet the single-digit-ms requirement in the measured configuration.

- Cost is architectural: autoregressive decoding, not raw parameter count.
- Average output length: 6.28 tokens $\Rightarrow$ 6–7 sequential decode steps per word.

Proposed Final Cascade

PROPOSED — motivated by experiments, not yet experimentally validated



Solid green = validated components. Dashed = proposed, not yet integrated.

What We Learned

- **Retrieval, not ranking, was the dominant bottleneck** — increasing ranker capacity (MLP, Cross-Encoder) did not improve, and sometimes hurt, end-to-end accuracy.
- **Generative normalization can bypass the candidate-coverage ceiling** — ByT5 reached 60.96% exact match on cases where retrieval could not even produce the gold candidate.
- **Representation and interaction are separate problems** — a from-scratch Cross-Encoder had interaction capacity but lacked pretrained spelling representation, and failed.
- **Generation's cost is architectural, not parametric** — the 66.76ms latency comes from autoregressive decoding, not from being a 300M-parameter model.
- **Next step is a cascade**: fast retrieval for the easy majority, generative fallback only for uncertain cases — motivated directly by these measurements.